



Este documento serve como um guia de referência técnico e prático para o desenvolvimento do projeto **labirinto_plus**. Ele estabelece as melhores práticas e fluxos de trabalho no Git Bash, com foco especial na resolução de conflitos e na gestão de pequenas variações de versões do motor gráfico Unity 6 entre diferentes ambientes de desenvolvimento (ex: Laboratório vs. Casa).



1. CLONANDO O PROJETO PELA PRIMEIRA VEZ

Quando você trabalha a partir de uma máquina nova (como um computador do laboratório) onde o projeto ainda não foi baixado, você não deve utilizar o comando `git fetch` ou `git pull` de imediato. É necessário realizar a clonagem completa do repositório para inicializar os metadados e os arquivos de trabalho.

1. Abra o **Git Bash** no seu sistema operacional.
2. Navegue até o diretório onde deseja centralizar seus projetos de desenvolvimento utilizando o comando `cd` :

```
cd "/c/Users/SeuUsuario/Documents/Unity Projects"
```

3. Execute o comando de clonagem apontando para a URL oficial do repositório no GitHub:

```
git clone https://github.com/shakmatton/labirinto_plus.git
```



Dica de Produtividade

Para colar comandos dentro do Git Bash, utilize o clique com o botão direito do mouse e selecione **Paste**, ou utilize o atalho de teclado nativo **Shift + Insert**.

4. **Importante:** Após a clonagem, você deve obrigatoriamente entrar na pasta criada pelo Git antes de executar qualquer outro comando de versionamento:

```
cd labirinto_plus
```



2. GERENCIAMENTO E FLUXO DE BRANCHES

Ao realizar a clonagem de um repositório, o Git baixa todo o histórico de desenvolvimento do servidor remoto, mas ativa localmente apenas a ramificação principal (geralmente chamada de `main` ou `master`).

Listando Ramificações disponíveis

Se você executar o comando simples de listagem, verá apenas a branch ativa na sua máquina:

```
git branch
```

Para visualizar a totalidade das ramificações guardadas com segurança no servidor do GitHub (tanto locais quanto remotas), adicione a flag de escopo total `-a`:

```
git branch -a
```

Alternando entre Ambientes de Trabalho

Para mudar o foco do seu desenvolvimento para uma branch específica (como a branch `inimigos`), você tem duas abordagens disponíveis no Git Bash:

Comando	Abordagem	Contexto de Uso
<code>git checkout inimigos</code>	Tradicional	Funciona universalmente em qualquer versão instalada do Git.
<code>git switch inimigos</code>	Moderna (Recomendada)	Introduzida em versões recentes para tornar a troca de contexto mais intuitiva.

🌟 Atalho para Novas Funcionalidades

Caso precise criar uma nova branch do zero e já alternar para ela imediatamente, utilize:

```
git checkout -b nome-da-nova-branch OU git switch -c nome-da-nova-branch
```

3. O ERRO "ABORTING / OVERWRITTEN BY CHECKOUT" NA UNITY

Um comportamento muito comum ao tentar alternar de branch em projetos que utilizam a Unity é o Git barrar a operação, exibindo uma mensagem de erro informando que arquivos locais seriam sobrescritos pela troca (*Your local changes to the following files would be overwritten by checkout*). Geralmente, os arquivos apontados estão em pastas como `ProjectSettings/` ou `Packages/`.

Por que esse fenômeno acontece?

Isso ocorre devido a pequenas discrepâncias nas versões da Unity instaladas nas máquinas de desenvolvimento. Por exemplo:

- Lab** Máquina do Laboratório executa a versão: **Unity 6000.4.0f1**
- Casa** Máquina Pessoal (Casa) executa a versão: **Unity 6000.4.1f1**

Assim que o projeto é identificado ou aberto pela Unity no laboratório, o motor gráfico reescreve automaticamente metadados essenciais de compatibilidade interna — como os arquivos `ProjectVersion.txt`, `manifest.json`,

`packages-lock.json` e configurações gráficas do pipeline (`UniversalRenderPipelineGlobalSettings.asset`). O Git rastreia essas modificações em tempo real e bloqueia a mudança de branch para evitar perda de dados acidental.

Como solucionar de forma limpa e segura

Dado que essas alterações foram geradas de forma sistêmica pela Unity e não representam códigos ou mecânicas implementadas manualmente por você, você pode descartar com segurança as modificações locais para liberar o caminho da troca de branch:

ATENÇÃO ALERTA

O comando abaixo realiza um descarte definitivo e irreversível das modificações não salvas na área de trabalho local. Certifique-se de que não há código autoral pendente antes de executá-lo.

```
# 1. Força o descarte de todas as modificações invisíveis da Unity e limpa o diretório
git reset --hard HEAD

# 2. Com a árvore de trabalho limpa, execute a alternância de branch sem bloqueios
git checkout inimigos
```

Nota: O Git compreenderá de forma inteligente que você está referenciando a branch remota existente no GitHub e criará uma cópia local sincronizada para o seu uso.

Abordagem Alternativa: O Uso do Stash

Se você estivesse desenvolvendo uma mecânica própria e quisesse trocar de branch sem perder o seu progresso atual nem fazer um commit inacabado, o comando ideal seria o uso do baú temporário:

`git stash` (guarda as alterações temporariamente) → `git checkout inimigos` (muda de branch) → `git stash pop` (resgata as alterações de volta quando retornar à branch de origem).

🛡️ 4. COMPATIBILIDADE DE VERSÕES E ROTINA DIÁRIA (CASA VS. LAB)

No desenvolvimento com Unity, alterações em versões estruturais profundas (grandes atualizações) costumam ser irreversíveis e quebram links e referências de prefabs e scripts. No entanto, na conjuntura atual do projeto **labirinto_plus**, as versões diferem apenas no dígito de **patch de correção de bugs** (`.0f1` vs `.1f1`).

Isso indica que o núcleo do motor gráfico e a estrutura dos metadados são idênticos. O risco de corrupção de arquivos é nulo e a Unity de casa (sendo ligeiramente mais recente) absorve as atualizações do laboratório de forma totalmente nativa.

Fluxo de Envio Padrão (Ao finalizar o dia no Laboratório)

Ao concluir suas tarefas de programação ou design na máquina do laboratório, feche o editor da Unity para liberar arquivos temporários e proceda com o envio regular do progresso:

```
# 1. Adiciona todos os novos arquivos e modificações geradas no dia
git add .

# 2. Documente o commit com uma mensagem descritiva clara
git commit -m "Feature: Avanço no sistema de inimigos e ajustes automáticos da Unity"

# 3. Envia os commits para a ramificação correspondente no GitHub
git push origin inimigos
```

Fluxo de Recebimento Padrão (Ao chegar em Casa)

No seu computador pessoal, abra o terminal do Git Bash dentro da pasta correta do repositório e execute os comandos para trazer as novidades do servidor externo:

```
# Garanta que está posicionado na branch correta de trabalho
git checkout inimigos

# Puxe as atualizações do servidor remoto para mesclá-las ao seu código local
git pull origin inimigos
```

⚠️ REGRA DE OURO PARA GRANDES ATUALIZAÇÕES FUTURAS

Se em algum momento futuro do projeto o laboratório ou algum integrante da equipe realizar um upgrade significativo na versão principal da Unity (ex: migrar para uma futura **Unity 6000.5.x** ou **Unity 7**), a compatibilidade automática deixará de existir.

Nesses cenários de divergência profunda de versões, **NUNCA utilize o comando generalista `git add .` ou `git add -A`**, pois os arquivos de configuração modificados pela nova Unity impedirão o projeto de abrir nos computadores com a versão anterior.

Adote a boa prática de adicionar de forma cirúrgica apenas os arquivos lógicos, scripts e assets criados por você:

```
# Exemplo: Adicionando apenas a pasta de scripts ou arquivos específicos
git add Assets/Scripts/
git add Assets/Sprites/MeuInimigo.png
```